Université Bretagne Sud
IUT de Vannes
Département Informatique — 2026/2027

BUT 2 — R3.07. SQL dans un langage de programmation

TD/TP 6

Procédures stockées et triggers

Contexte général

Lors des TP précédents, l'application bancaire a été développée en faisant évoluer progressivement l'accès aux données :

- accès direct via **JDBC** ;
- utilisation de **JPA** et de l'ORM ;
- structuration de l'accès aux données avec le **patron Repository**.

Dans tous ces cas, la logique métier était principalement implémentée **côté application Java**.

Dans ce dernier TP, une partie de cette logique va être déplacée **côté base de données**, à l'aide des fonctionnalités avancées des serveurs de bases de données (SGBD), dans notre cas PostgreSQL :

- les **procédures stockées** ;
- et les **triggers**.

La base de données PostgreSQL sera exécutée dans un conteneur Docker, afin de garantir un environnement reproductible. Nous allons écrire ces deux fonctionnalités avec le langage PL/pgSQL (*Procedural Language PostgreSQL*). Un mini-tutoriel est donné à la fin du sujet.

Procédures stockées

Une procédure stockée est un programme stocké et exécuté directement par le SGBD (dans notre cas, PostgreSQL). Elle permet :

- de regrouper plusieurs instructions SQL dans une seule unité logique ;
- de centraliser certaines règles métier ;
- de réduire les échanges entre l'application et la base de données ;
- d'améliorer la sécurité en contrôlant l'accès aux données.

Dans PostgreSQL, les procédures et fonctions peuvent être écrites en `PL/pgSQL` et invoquées depuis une application Java via JDBC ou JPA.

Triggers

Un trigger est un mécanisme permettant d'exécuter automatiquement une action lorsqu'un événement se produit sur une table (INSERT, UPDATE, DELETE, ...).

Les triggers sont principalement utilisés pour :

— garantir des règles d'intégrité complexes ;
— tracer des modifications (journalisation) ;
— maintenir la cohérence des données ;
— appliquer des règles indépendantes de l'application.

Un trigger est associé à une table et repose sur une fonction déclenchée automatiquement.

Schéma de la base de données

On reprend les tables de l'application bancaire développée précédemment :

— `Client(numClient, nomClient, prenomClient, ...)`
— `Compte(numCompte, solde, typeCompte)`
— `Compte_Client(numClient, numCompte)`
— `Agent(numAgent, nomAgent, ...)`
— `Agence(numAgence, telAgence, ...)`

Exercice 1 – Procédures stockées

On souhaite implémenter une opération de virement bancaire entre deux comptes.

Travail demandé

a) Créer une procédure stockée `virement` prenant en paramètres :
   — l'identifiant du compte source ;
   — l'identifiant du compte destination ;
   — le montant du virement.

b) La procédure devra :
   — vérifier que le compte source dispose d'un solde suffisant ;
   — débiter le compte source ;
   — créditer le compte destination ;
   — garantir l'atomicité de l'opération.

c) Tester la procédure directement dans PostgreSQL.

d) Expliquer l'intérêt d'implémenter cette logique sous forme de procédure stockée plutôt que côté application Java.

e) Que se passe-t-il si le compte **destination** n'existe pas ? Testez avec un numéro inexistant, puis regardez le solde du compte source. Corrigez la procédure (indice : la variable `FOUND` après un `UPDATE`).

f) Appelez la procédure depuis votre application Java :

```
try (PreparedStatement ps = c.prepareStatement("CALL virement(?, ?, CAST(? AS REAL))")) {
    ps.setInt(1, src);
    ps.setInt(2, dst);
    ps.setDouble(3, montant);
    ps.execute();
} catch (SQLException e) {
    System.err.println(e.getSQLState() + " : " + e.getMessage());
}
```

Provoquez un solde insuffisant : quel `SQLState` reçoit Java ? Transformez cette erreur en exception métier `SoldeInsuffisantException`. Pourquoi le `CAST` est-il nécessaire (essayez sans) ?

La correction est disponible sur Moodle. Pour bien en comprendre les détails, nous vous invitons à lire attentivement le sujet jusqu'à la fin.

Exercice 2 – Triggers

On souhaite tracer automatiquement les opérations effectuées sur les comptes bancaires.

Travail demandé

a) Créer une table `HistoriqueCompte` contenant :
   — l'identifiant du compte ;
   — l'ancienne valeur du solde ;
   — la nouvelle valeur du solde ;
   — la date de modification.

b) Créer une fonction de trigger permettant d'insérer automatiquement une ligne dans `HistoriqueCompte` lors d'une modification du solde d'un compte.

c) Associer cette fonction à un trigger déclenché lors d'un `UPDATE` sur la table `Compte`.

d) Tester le trigger à l'aide :
   — d'une mise à jour SQL directe ;
   — de la procédure stockée du premier exercice.

e) Expliquer l'intérêt d'un trigger par rapport à une journalisation réalisée côté application.

La correction est disponible sur Moodle. Pour bien en comprendre les détails, nous vous invitons à lire attentivement le sujet jusqu'à la fin.

Exercice 3 – Curseurs : relevé de compte

On souhaite produire le relevé d'un compte : ses opérations dans l'ordre chronologique, avec le **solde cumulé** après chaque opération (un dépôt augmente le cumul, un retrait le diminue).

Travail demandé

a) Écrire une fonction `releve_compte(p_num INT)` qui retourne une table `(date_op DATE, type_op VARCHAR, montant_op REAL, cumul REAL)`. Elle doit parcourir les opérations du compte avec un **curseur explicite** (`OPEN`, `FETCH`, `CLOSE`) et renvoyer chaque ligne avec `RETURN NEXT`.

b) Lever une exception si le compte n'existe pas.

c) Tester : `SELECT * FROM releve_compte(1);`

d) Obtenir le même résultat avec **une seule requête SQL**, sans curseur, grâce à une fonction de fenêtre : `sum(...) OVER (ORDER BY dateOperation, numOperation)`. Quelle version préférez-vous, et pourquoi ?

e) Depuis Java, afficher le relevé avec un `PreparedStatement` sur `SELECT * FROM releve_compte(?)`.

Rendu attendu

— Les scripts SQL (procédures, triggers, tests) sont donnés sur Moodle. Pas besoin de les rendre ;
— Résultats des tests (captures d'écran) ;
— Réponses argumentées aux questions dans un fichier Markdown.

Exemples de procédures stockées et triggers

1. Exemples de procédures stockées (logique métier explicite)

Les procédures stockées sont idéales pour encapsuler des règles métier critiques, transactionnelles et réutilisables par toutes les applications clientes.

1. **Clôture d'un compte bancaire**

   Procédure : `cloturer_compte(numCompte)`

   — Vérifie que le solde est nul.
   — Empêche la clôture si le compte est débiteur.
   — Supprime ou archive le compte.

   Pourquoi une procédure ? Règle métier critique, transactionnelle, commune à toutes les applications clientes.
2. **Ouverture d'un compte**

   Procédure : `ouvrir_compte(typeCompte, soldeInitial)`

   — Crée un nouveau compte.
   — Applique des règles selon le type de compte (minimum initial, découvert autorisé).
   — Initialise correctement les données.

   Pourquoi une procédure ? Centralise les règles d'ouverture indépendamment du code Java.
3. **Changement de type de compte**

   Procédure : `changer_type_compte(numCompte, nouveauType)`

   — Vérifie la compatibilité du type avec le solde actuel.
   — Empêche certains changements (ex : compte épargne → compte courant).

   Pourquoi une procédure ? Règles complexes dépendantes des données existantes.
4. **Calcul et versement des intérêts**

   Procédure : `verser_interets()`

   — Parcourt tous les comptes épargne.
   — Calcule les intérêts.
   — Met à jour les soldes.

   Pourquoi une procédure ? Traitement batch, exécuté périodiquement côté base (cron, tâche planifiée).
5. **Virement groupé**

   Procédure : `virement_multiple(listeComptes, montant)`

   — Effectue plusieurs virements dans une seule transaction.
   — Annule tout si une seule opération échoue.

   Pourquoi une procédure ? Garantie d'atomicité sur plusieurs opérations.

2. Exemples de triggers (règles automatiques et transverses)

Les triggers sont utilisés pour appliquer des règles automatiques, transverses et non contournables.

1. **Interdiction de solde négatif**

   Trigger : `before_update_solde`

— Vérifie qu'un solde ne devient pas négatif.
— Bloque l'opération automatiquement.

Pourquoi un trigger ? Règle d'intégrité qui doit s'appliquer même en cas d'UPDATE SQL direct.

2. **Journalisation des opérations**

   Trigger : `after_update_compte`

   — Enregistre toute modification de solde.
   — Sert à l'audit et à la traçabilité.

   Pourquoi un trigger ? Indépendant de l'application, impossible à contourner.

3. **Détection d'opérations suspectes**

   Trigger : `after_update_solde`

   — Détecte des montants inhabituels.
   — Insère une alerte dans une table dédiée.

   Pourquoi un trigger ? Surveillance automatique et continue.

4. **Mise à jour d'un solde global**

   Trigger : `after_update_compte`

   — Met à jour le solde total d'une agence.
   — Maintient des données agrégées cohérentes.

   Pourquoi un trigger ? Évite des calculs coûteux répétés côté application.

5. **Historisation des comptes supprimés**

   Trigger : `before_delete_compte`

   — Archive les informations du compte supprimé.
   — Permet une récupération ou un audit ultérieur.

   Pourquoi un trigger ? Traçabilité forte, aucune suppression silencieuse.

Message clé

— **Les procédures stockées** sont adaptées aux règles métier explicites et transactionnelles ;
— **Les triggers** sont adaptés aux règles automatiques, transverses et non contournables.

Mini-tutoriel : bases du PL/pgSQL

PL/pgSQL (Procedural Language PostgreSQL) est un langage procédural permettant d'écrire des programmes exécutés directement par PostgreSQL. Il est utilisé pour définir des **procédures stockées** et des **triggers**.

Ce tutoriel présente uniquement les éléments nécessaires à la réalisation de ce TP.

Structure générale d'une procédure

Une procédure stockée PostgreSQL se définit avec l'instruction `CREATE PROCEDURE` et contient un bloc `BEGIN ... END`.

```
CREATE OR REPLACE PROCEDURE nom_procedure(paramètres)
LANGUAGE plpgsql
AS $$
BEGIN
    -- instructions SQL
END;
$$;
```

Une procédure peut être appelée avec l'instruction `CALL`.

```
CALL nom_procedure(...);
```

Déclaration de variables

Les variables sont déclarées dans une section `DECLARE`.

```
DECLARE
    solde_source REAL;
```

Elles peuvent ensuite être utilisées dans le corps de la procédure.

Récupération d'un résultat dans une variable

L'instruction `SELECT ... INTO` permet de stocker le résultat d'une requête SQL dans une variable.

```
SELECT solde INTO solde_source
FROM Compte
WHERE numCompte = 1001;
```

Conditions

Les structures conditionnelles utilisent `IF / THEN / END IF`.

```
IF solde_source < 0 THEN
    RAISE EXCEPTION 'Solde négatif';
END IF;
```

Gestion des erreurs

L'instruction `RAISE EXCEPTION` permet d'arrêter l'exécution de la procédure et d'afficher un message d'erreur.

```
RAISE EXCEPTION 'Erreur : solde insuffisant';
```

Transactions

Dans PostgreSQL, une procédure stockée s'exécute par défaut dans une transaction. Si une exception est levée, toutes les modifications sont annulées.

Cela garantit l'atomicité des opérations sensibles comme un virement bancaire.

Curseurs

Un curseur parcourt le résultat d'une requête **ligne par ligne**. On le déclare, on l'ouvre, on lit chaque ligne avec `FETCH`, et on s'arrête quand `FOUND` devient faux.

```
DECLARE
    cur CURSOR FOR SELECT numCompte, solde FROM Compte ORDER BY numCompte;
    ligne RECORD;
BEGIN
    OPEN cur;
    LOOP
        FETCH cur INTO ligne;
        EXIT WHEN NOT FOUND;
        RAISE NOTICE 'Compte % : %', ligne.numCompte, ligne.solde;
    END LOOP;
    CLOSE cur;
END;
```

La forme `FOR ligne IN SELECT ... LOOP ... END LOOP;` utilise un curseur implicite. Une fonction qui renvoie plusieurs lignes se déclare `RETURNS TABLE(...)` et renvoie chaque ligne avec `RETURN NEXT`. Attention : un traitement ligne par ligne est souvent plus lent qu'une requête SQL ensembliste équivalente.

Fonctions et triggers

Un trigger repose sur une **fonction** qui retourne `TRIGGER`.

```
CREATE OR REPLACE FUNCTION nom_fonction()
RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
BEGIN
    -- traitement automatique
    RETURN NEW;
END;
$$;
```

Dans une fonction de trigger :

- `OLD` représente l'ancienne ligne dans la table ;
- `NEW` représente la nouvelle ligne.

Création d'un trigger

Un trigger est associé à une table et à un événement.

```
CREATE TRIGGER nom_trigger
AFTER UPDATE
ON Compte
FOR EACH ROW
EXECUTE FUNCTION nom_fonction();
```

Le code de la fonction est exécuté automatiquement lors de l'événement concerné.